

ShoeShop — Viva Questions & Answers

E-Commerce Web Application — Next.js 16, React 19, TypeScript, Tailwind CSS 4, eSewa Payment

A. Introduction / Project Overview

Q1. What is your project?

A complete e-commerce web application called **ShoeShop** for buying sneakers and footwear online. It has a product catalog, search, product detail pages, a shopping cart, user authentication, an admin dashboard to manage products and view orders, image upload, and online payment via the **eSewa** payment gateway.

Q2. What is the tech stack used?

- **Next.js 16** (App Router) — React framework for SSR/SSG and API routes
- **React 19** — UI library
- **TypeScript** — type safety
- **Tailwind CSS 4** — styling
- **shadcn/ui + Radix UI** — reusable UI components
- **eSewa Payment Gateway** — online payment
- **JSON files** — data storage (no external DB)

Q3. What are the main features?

1. Product catalog with category filter and search
2. Product detail pages with quantity selection
3. Shopping cart with add/remove/update quantity
4. Auth: register, login, logout, roles (admin/customer)
5. Admin dashboard: add products, upload images, view orders
6. eSewa payment integration (sandbox)
7. Payment success/cancel pages with verification
8. Dark theme with animated 3D background, responsive design

Q4. Who are the users of the system?

Two types: **customers** (browse, add to cart, pay) and **admin** (add products, view orders).

Q5. What problem does it solve?

It digitizes shoe retail — customers can browse, search, and buy shoes online from home, and the shop owner can manage products and track orders through an admin panel instead of manual/offline sales.

B. Architecture & Flow

Q6. Explain the overall architecture.

Client-side pages (React) talk to Next.js **Route Handlers** (/api/*) which perform server-side logic (auth, file I/O). Data is stored in JSON files in /data. Payment is done by redirecting to eSewa, then verified via a status API call.

Q7. What is the flow of placing an order?

1. User adds items to cart (stored in localStorage via Context).
2. At checkout, the app calls /api/esewa/initiate which builds the payment form + signature.
3. The user is redirected to eSewa; pays in the eSewa wallet.
4. eSewa redirects back to /payment/success?data=...
5. The success page calls /api/esewa/verify to confirm with eSewa's server.
6. If verified, the order is saved to orders.json and the cart is cleared.

Q8. How does the admin panel work?

/admin first calls /api/auth/me to check the user's role. If not admin, it redirects to login. If admin, it loads orders via /api/orders and provides a form to add products (with image upload via /api/upload).

Q9. How are server and client components separated?

Components that need browser features (state, effects, event handlers) are marked "use client" (e.g., Header, CartContext, checkout page). Static/server components like the homepage sections (Hero, PromoBanner) are server components rendered on the server.

C. Next.js / Routing

Q10. What is the App Router?

Next.js's modern routing system where folders under /app map to URLs. app/page.tsx → /, app/shop/page.tsx → /shop, app/product/[productId]/page.tsx → /product/1, etc. Dynamic segments use [brackets].

Q11. What are Route Handlers?

API endpoints defined in app/api/.../route.ts files. They export GET, POST, etc. functions that return JSON. Examples: /api/products, /api/auth/login, /api/orders.

Q12. How do you read the dynamic route param?

In app/product/[productId]/page.tsx, useParams() is used to get productId, then it fetches products and finds the matching one.

Q13. What is a Client Component vs Server Component?

Server components render on the server (no JS sent to client, better performance/SEO). Client components ("use client") run in the browser, needed for interactivity like state and useEffect.

Q14. What is Next.js's Image component?

next/image provides automatic image optimization, lazy loading, and responsive sizing (used on product detail page with width, height, priority).

D. State Management (Cart)

Q15. How is the cart state managed?

Using **React Context** (CartProvider in context/CartContext.tsx) combined with useState, and **persisted to localStorage** so the cart survives page refreshes.

Q16. Why Context + localStorage?

Context gives a global state shared across components (Header badge, cart page) without prop-drilling. localStorage persists it across sessions — two useEffects sync state ↔ localStorage.

Q17. What functions does the cart expose?

addToCart, removeFromCart, clearCart, updateQuantity, plus the cart array itself.

Q18. How is quantity handled when adding an existing item?

If the item ID already exists in the cart, addToCart increments its quantity by 1 instead of duplicating the entry.

Q19. What is the useCart hook and why does it throw?

It's a custom hook that returns the context. It throws an error if used outside the CartProvider — a safety guard to catch misuse.

Q20. Why is cart stored in localStorage vulnerable?

Because localStorage is client-side and can be modified by the user — it's only for convenience, not trusted for security. The server re-validates critical data (payment amount) during eSewa verification.

E. Authentication & Security

Q21. How is authentication implemented?

Custom session auth. On login/register, the server creates a signed token (HMAC-SHA256 over a JSON payload with expiry) and sets it in an **httpOnly cookie** named shoeshop_session.

Q22. How are passwords stored?

Never plaintext. Each password is hashed with **SHA-256** along with a random 16-byte salt, stored as `salt:hash` (see `hashPassword` in `lib/auth.ts`). Verification recomputes the hash and compares.

Q23. Why use a salt?

So identical passwords produce different hashes, defending against rainbow-table/precomputed-hash attacks.

Q24. What is HMAC and why is it used for the session token?

HMAC-SHA256 is a keyed hash. The token's payload is signed with a server secret (`AUTH_SECRET`), so if anyone modifies the payload (e.g., changes role to admin), the signature won't match and the session is rejected.

Q25. What cookie attributes are set and why?

- `httpOnly: true` — JavaScript can't read the cookie, preventing XSS token theft.
- `sameSite: "lax"` — mitigates CSRF.
- `secure: true` in production — only sent over HTTPS.
- `maxAge: 7 days` — session expires after a week.

Q26. How is the session validated on each request?

`verifySession` splits `body.signature`, recomputes the HMAC, compares, and checks the exp timestamp. If expired or tampered, it returns null.

Q27. How is admin protected?

`requireAdmin()` in `lib/auth.ts` reads the session and throws if the user isn't role admin. It's called in `/api/products` (POST), `/api/orders` (GET), and `/api/upload`. The admin page also checks `/api/auth/me` client-side.

Q28. What is role-based access control?

Users have a role field ("admin" or "customer"). Certain operations (adding products, uploading images, viewing all orders) are restricted to admins only.

Q29. What is a security flaw in this auth approach?

It's a custom scheme; using a well-tested library (NextAuth, JWT, or bcrypt for passwords) is safer in production. Also the fallback `AUTH_SECRET` in code should never be committed — it's in `.env.local` in production.

Q30. What is XSS?

Cross-Site Scripting — injecting malicious client-side scripts. Mitigated here with `httpOnly` cookies so scripts can't steal the session.

Q31. What is CSRF?

Cross-Site Request Forgery — tricking a user's browser into sending authenticated requests. Mitigated by sameSite: "lax" cookies and server-side validation of required fields.

F. Payment (eSewa) Integration

Q32. What is eSewa?

A popular Nepalese digital wallet / payment gateway. The app integrates eSewa's **EPay v2** API.

Q33. Explain the initiate step (/api/esewa/initiate).

It receives amount, tax, total, transaction UUID, charges, and product name; builds an HMAC-SHA256 **signature** over total_amount, transaction_uuid, product_code using the secret key; returns the hidden form fields and eSewa's payment URL.

Q34. Why the signature?

eSewa verifies the signature to ensure the amount and transaction data weren't tampered with in transit. It's computed server-side with the merchant secret key, never exposed to the client.

Q35. How does the app redirect to eSewa?

Client-side, the checkout page dynamically creates an HTML `<form method="POST">` with hidden inputs (the formData from initiate) and calls `formEl.submit()`, redirecting the browser to eSewa's sandbox URL.

Q36. Explain the verify step (/api/esewa/verify).

eSewa redirects back with a base64-encoded data param. The app decodes it, then calls eSewa's **status API** with product_code, total_amount, transaction_uuid. Payment is "verified" only if the returned status === "COMPLETE".

Q37. Why verify server-to-server?

Because the redirect data could be forged. The status check talks directly to eSewa's server to confirm the transaction really completed — this is the source of truth.

Q38. What is transaction_uuid for?

A unique ID per transaction (generated as `Date.now() + random`), used to prevent duplicate payments and to match the order.

Q39. What is ref_id?

The eSewa transaction reference ID returned after successful payment — stored with the order and shown to the user.

Q40. What are sandbox vs production endpoints?

The code uses sandbox URLs (`rc-epay.esewa.com.np`) and test product code `EPAYTEST`. In production you switch to `esewa.com.np/api/epay/v2/form` with your live merchant credentials.

Q41. What sandbox test credentials exist?

eSewa ID 9806800001-9806800005, password `Nepal@123`, OTP 123456. Admin login: `admin@shoeshop.com / admin123`.

Q42. What happens if payment fails or is cancelled?

eSewa redirects to `/payment/cancel`, showing a cancellation message; no order is created.

Q43. What is the tax and shipping logic?

- Shipping: **Rs 199**, free if subtotal > **Rs 5000**.
- Tax: **8%** of subtotal.
- Total = subtotal + shipping + tax.

Q44. Where is pending payment data stored?

In `sessionStorage` under `esewa_pending` (transaction UUID, total amount, customer info) so the success page can reconstruct and save the order.

G. Backend / Data Storage

Q45. Where is data stored?

In JSON files: `data/products.json`, `data/users.json`, `data/orders.json`.

Q46. How is data read/written?

Via `lib/store.ts` using Node's `fs/promises` — `readJsonFile` reads and `JSON.parse`s a file; `writeJsonFile` stringifies and writes it. They run only on the server (in Route Handlers).

Q47. What are the advantages/disadvantages of JSON files vs a database?

Advantages: simple, zero setup, good for prototypes/small data. **Disadvantages:** no concurrency safety, slow for large data, no querying/indexing, not production-scale. A real DB (PostgreSQL/MySQL/MongoDB) is needed for production.

Q48. Why is `process.cwd()` used?

`process.cwd()` returns the project root so file paths are absolute and correct regardless of where the command runs.

Q49. How are new IDs generated?

The route reads the file, takes `Math.max(...ids) + 1`, or starts at 1 if empty.

Q50. How does the image upload work (/api/upload)?

Admin sends a file via FormData. The route validates: admin only, file type must be JPEG/PNG/WebP/GIF, size $\leq$ 5MB. It generates a unique filename (timestamp-randomhex.ext), saves to public/uploads, and returns the public URL /uploads/...

Q51. Why generate random filenames?

To prevent filename collisions and path-traversal attacks (safe, unpredictable names).

Q52. What is the /api/auth/me endpoint?

A GET route that returns the current logged-in user by reading and verifying the session cookie — used by the Header and admin page.

H. Frontend / UI / React

Q53. How is the product list filtered?

In ShopClient.tsx, useMemo filters products by selected **category** and a **search query** against the product name, both case-insensitive.

Q54. What is useMemo?

A React hook that caches/memoizes a computed value so it only recomputes when its dependencies (products, category, query) change — improving performance.

Q55. How does category selection update the URL?

setCategoryParam uses window.history.replaceState to add/remove ?category=... without reloading the page, keeping the filter shareable/bookmarkable.

Q56. What UI library components are used?

shadcn/ui-style components built on Radix UI: Button, Card, Input, Textarea, Select, Badge, Separator, using clsx + tailwind-merge via the cn utility.

Q57. What is the cn utility (lib/utils.ts)?

A helper that combines clsx (conditional class names) and tailwind-merge (resolves conflicting Tailwind classes) to build class strings.

Q58. What is a Context Provider and how is it used here?

CartProvider wraps the whole app in layout.tsx, giving every component access to the cart via useCart().

Q59. Why is Suspense used on the payment success page?

Because useSearchParams() must be wrapped in Suspense during static rendering in Next.js — the fallback shows "Loading..." while the component waits.

Q60. What is the animated background component?

`components/layout/Background.tsx` renders a fixed animated 3D/gradient background behind the content (relative `z-10` on `main`) for the dark theme.

I. Styling

Q61. What CSS framework is used?

Tailwind CSS v4, configured with `tailwind.config.js`, plus `tailwindcss-animate` for animations and `globals.css` for global styles/theme tokens.

Q62. How is the design made responsive?

Using responsive prefixes: `grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4`, `hidden md:flex`, `lg:hidden`, `sm:block`, etc.

Q63. What is the color theme?

A dark theme using CSS variables mapped to Tailwind tokens like `bg-background`, `text-foreground`, `bg-primary`, `text-muted-foreground`, `border-border` — allowing easy theme switching.

J. Testing / Deployment / Improvements

Q64. How do you run the project?

`npm i` then `npm run dev` (or `npm run build + npm start` for production). Lint with `npm run lint`.

Q65. What are the project scripts?

`dev`, `build`, `start`, `lint` in `package.json`.

Q66. Are there tests?

No automated test suite is included; testing would be manual or with Jest/Testing Library added later.

Q67. What environment variables are used?

`AUTH_SECRET`, `ESEWA_PAYMENT_URL`, `ESEWA_STATUS_URL`, `ESEWA_PRODUCT_CODE`, `ESEWA_SECRET_KEY`, `ESEWA_SUCCESS_URL`, `ESEWA_FAILURE_URL`, `NEXT_PUBLIC_APP_URL` — stored in `.env.local`.

Q68. What improvements would you make?

- Use a real database (PostgreSQL/Prisma) and proper ORM.
- Use bcrypt for password hashing and NextAuth/JWT libraries.
- Add product editing/deleting, order status management, stock/inventory.
- Add email notifications, order history for customers, wishlist persistence.
- Rate limiting on API, server-side input validation (e.g., zod).
- Move to production eSewa credentials and a proper hosting platform (Vercel).

Q69. What are the biggest security concerns in the current implementation?

- Fallback secret key and hardcoded sandbox credentials in code/README.
- Custom auth instead of a proven library.
- No rate limiting on login (brute force).
- Client-side total is re-sent; must always trust eSewa's verified amount server-side.

K. Conceptual / General

Q70. What is the difference between sessionStorage and localStorage?

localStorage persists across tabs and sessions; sessionStorage is cleared when the tab/session closes. Cart uses localStorage; pending payment uses sessionStorage.

Q71. What is HTTP-only cookie and why is it important?

A cookie flag preventing JavaScript access, so tokens can't be stolen via XSS — protecting session tokens.

Q72. What is the difference between client-side and server-side rendering?

SSR renders HTML on the server (better SEO/first paint); CSR renders in the browser via JS. Next.js combines both (SSR/SSG for server components, CSR for interactive ones).

Q73. What is an API endpoint vs a server action?

Route Handlers (/api/*) are REST-style HTTP endpoints returning JSON; Next.js Server Actions are alternative functions callable directly. This project uses Route Handlers.

Q74. Why TypeScript?

Static typing catches bugs at compile time, improves autocomplete and code maintainability (interfaces like Product, CartItem, Order).

Q75. Explain the full file-to-screen journey for /shop.

app/shop/page.tsx renders <ShopClient />, which fetches /api/products on mount → that Route Handler reads products.json → returns JSON → the client filters by category/search and renders ProductCards.

L. Sample "Surprise" / Practical Questions

Q: Where would you add a "delete product" feature?

Add a DELETE handler in `/api/products/route.ts` guarded by `requireAdmin()`, and a button in the admin panel.

Q: How would you prevent two users from writing `orders.json` at the same time?

Add a simple file lock/queue, or move to a real database with atomic transactions.

Q: Why is the total amount checked against eSewa's status response?

So a user can't tamper with the client-side amount and pay less — the server compares its own record with eSewa's returned `total_amount`.